

Greed is good



The point is, ladies and gentleman, that greed, for lack of a better word, is good. Greed is right, greed works. Greed clarifies, cuts through, and captures the essence of the evolutionary spirit. Greed, in all of its forms; greed for life, for money, for love, for knowledge has marked the upward surge of mankind. And greed, you mark my words, will not only save Teldar Paper, but that other malfunctioning corporation called the USA. Thank you very much.

Gordon Gekko, Wall Street

Migliora la tua carriera

Scopri le tue potenzialità di guadagno, trova lavori da sogno e condividi approfondimenti su lavoro e vita privata in forma anonima.

[Accedi/iscriviti](#)

Aziende seguite

Non lasciarti sfuggire opportunità e informazioni privilegiate seguendo le aziende dove vorresti lavorare.

[Cerca aziende](#)

Lavori salvati

Ricevi suggerimenti e aggiornamenti personalizzati avviando le tue ricerche.

[Cerca offerte di lavoro](#)

Colloquio per Software Engineer

27 ott 2025 ...

Candidato anonimo a colloquio

— Offerta rifiutata Esperienza positiva Colloquio difficile

Candidatura

Ho sostenuto un colloquio presso Google

Colloquio

Online Assessment (OA): Two algorithmic coding questions on platforms similar to HackerRank or CodeSignal. Topics included data structures (arrays, strings, graphs) and complexity analysis. The key challenge was writing clean, efficient code within 90 minutes. Technical Phone Interview: Conducted by a Google engineer via Google Meet. Focused on problem-solving and communication – one medium-level LeetCode-style problem plus follow-up optimizations. The interviewer expected clear thinking, complexity trade-offs, and edge case handling. Onsite (or Virtual Onsite): Comprised of 3–4 rounds: Algorithms & Data Structures: Dynamic programming, graph traversal, and binary trees. System Design (for experienced candidates): High-level design of scalable systems and trade-offs. Behavioral (“Googliness”): Collaboration, learning attitude, handling ambiguity, and leadership without authority. Hiring Committee & Offer: Final evaluation based on technical performance, problem-solving depth, and teamwork culture fit. Takes about 1–2 weeks.

Domande di colloquio [1]

Domanda 1

Arrays and hash maps Recursion and dynamic programming Graphs and shortest path Design patterns for embedded or performance-oriented systems Coding in C++ / Python / Java (depending on preference)

[Rispondi alla domanda →](#)

Algoritmi greedy

Quando si applicano?

Problemi di ottimizzazione

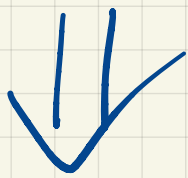
Programmazione
dinamica

- bottom-up
- sottostrutture
ottime
- evita le
ripetizioni

//

Tecniche
Greedy
(ingorde,
evide)

- sottostrutture
ottime
- opere con
"scelte locali"
scelte locali
ottime



Equazione di
programmazione
dinamica



costruire la
soluzione
per "aggiunte"
con **SCELTA**
ingorda



ne
cessite
una
dimostrazione
formale

stesso problema
risolto da
PD e Greedy

può
migliorare
il tempo

rispetto ad algoritmo
di PD!

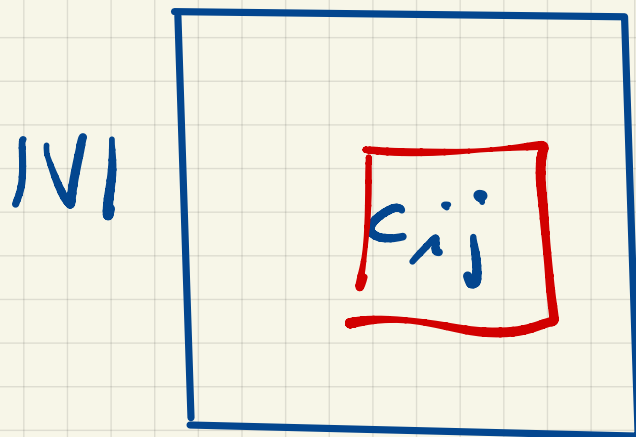
Applicazioni tipiche del Greedy -

- Minimo albero di copertura
(rete di telecomunicazioni)
- problema dei cammini minimi
- pianificazione delle attività
- problema del RESTO
"in monete"
minimo numero di pezzi
per dare il RESTO

$\text{tempo} \geq \text{spazio occupato}$

↓
PD

riempimento
delle matrice
 $|V|$

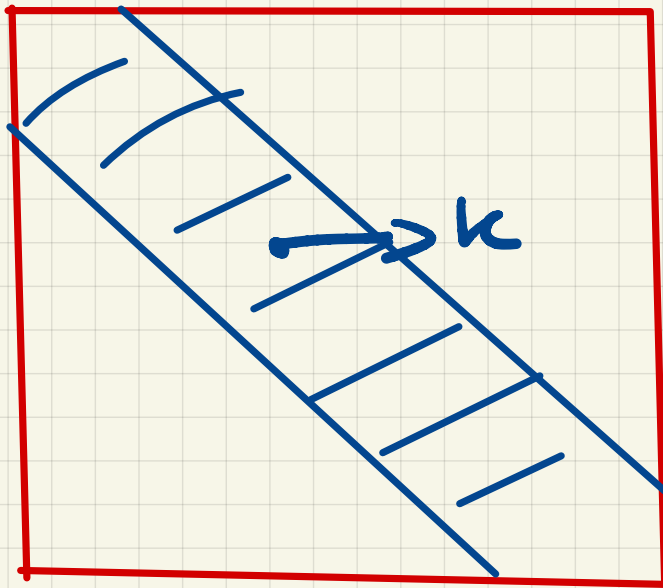


$O(|V|^2)$

Metodo anche in PD

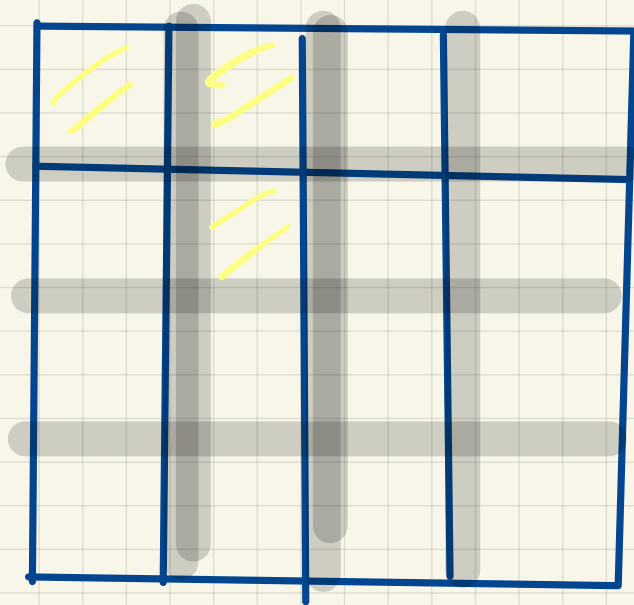
per non riempire tutte

le matrici



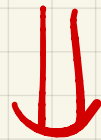
→ criterio
della
banda

$$O(n \cdot k)$$



tecnica
dei
4 Russi

"single source shortest-path"



Greedy

PROBLEMA Change-Making

come dare un resto con
il minimo n° di monete
dato un SISTEMA di MONETE
o COIN-SYSTEM

esempio

- 1 €
- 50 cent
- 20 cent
- 5 cent
- 10 cent

¢

coin -
system

Canonical Coin Systems for Change-Making Problems

Xuan Cai

Department of Computer Science and Engineering, Shanghai Jiao Tong University

Shanghai 200240, China

Email: caixuanfire@sjtu.edu.cn

00v2 [cs.DS] 21 Mar 2009

Abstract—The Change-Making Problem is to represent a given value with the fewest coins under a given coin system. As a variation of the knapsack problem, it is known to be NP-hard. Nevertheless, in most real money systems, the greedy algorithm yields optimal solutions. In this paper, we study what type of coin systems that guarantee the optimality of the greedy algorithm. We provide new proofs for a sufficient and necessary condition for the so-called *canonical* coin systems with four or five types of coins, and a sufficient condition for non-canonical coin systems, respectively. Moreover, we present an $O(m^2)$ algorithm that decides whether a tight coin system is canonical.

I. INTRODUCTION

The Change-Making Problem comes from the following scenario: in a shopping mall, the cashier needs to make change for many values of money based on some *coin system*² $\$ = \langle c_1, c_2, \dots, c_m \rangle$ with $1 = c_1 < c_2 < \dots < c_m$, where c_i denotes the value of the i -th type of coin in $\$$. For example, the cent, nickel, dime and quarter are four types of US coins, and the corresponding coin system is $\$ = \langle 1, 5, 10, 25 \rangle$. Since the

approach, can produce the optimal solutions for many practical instances, especially canonical coin systems.

Definition 1: A coin system $\$$ is canonical if $|\text{GRD}_{\$}(x)| = |\text{OPT}_{\$}(x)|$ for all x .

For example, the coin system $\$ = \langle 1, 5, 10, 25 \rangle$ is canonical. Accordingly, the cashier can easily create the optimal solution by repeatedly taking the largest coin whose value is no larger than the remaining amount.

Definition 2: A coin system $\$$ is non-canonical if there is an x with $|\text{GRD}_{\$}(x)| > |\text{OPT}_{\$}(x)|$, and such x is called a counterexample of $\$$.

Definition 3: A coin system $\$ = \langle 1, c_2, \dots, c_m \rangle$ is tight if it has no counterexample smaller than c_m .

For example, both $\$_1 = \langle 1, 7, 10, 11 \rangle$ and $\$_2 = \langle 1, 7, 10, 50 \rangle$ are non-canonical, and $\$_1$ is tight but $\$_2$ is not. It is easy to verify that 14 is the counterexample for them, i.e., $\text{GRD}_{\$_1}(14) = \text{GRD}_{\$_2}(14) = (3, 0, 0, 1)$ and $\text{OPT}_{\$_1}(14) = \text{OPT}_{\$_2}(14) = (0, 2, 0, 0)$.

Quando Greedy funziona
e quando non funziona?

es. Knapsack 0/1
(zaino)

VALORE

PESO

oggetto 1 e_1 100

40

oggetto 2 e_2 70

20

oggetto 3 e_3 60

30

CAPACITÀ

ZAINO

$\bar{e}$ $C=50$

Greedy o altro sceglie
prima e_1 e "satura"

lo ZAINO

valore totalizzato è 100

peso $40 < 50$ ma

non può aggiungere π
nella -

SOLUZIONE OTTIMA È

$I' \subseteq \{e_2, e_3\}$ valore
massimo $\bar{c}$

$$70 + 60 = 130$$

pesi sommati $20 + 30 = 50$

ci stanno nello zaino!

Non posso applicare

Greedy!

CONTRO-
esempio

Greedy funziona nel
Knapsack frazionario!

(posso prendere anche
frazioni dell'elemento)
più copie dell'
elemento!

Greedy costruisce soluzione
ottima per "aggiunta"
devo sapere quando posso
costruire una "soluzione
ottima parziale" e
partire da una soluzione
S (parziale) operando
l' UNIONE insiemistica

$\subseteq \overline{e}$

$S \cup \{e\} ?$

Per fare questo si
introducono gli INSIEMI
che godono di 2 PROPRIETÀ

(1) Ereditarietà

(2) Scambio

una famiglia di

INSIEMI che gode
della proprietà (1) e

(2) $\Rightarrow$ MATROIDE
e un input e greedy
is good!

• $E =$ insieme finito
di elementi

(es. coins,
oggetto dello
zaino, etc.)

• $F =$ collezione
di sottoinsiemi di E ,

FAMIGLIA

$$\downarrow$$
$$(F \subseteq 2^E)$$

es. famiglia
come

coins,

sottoinsiemi di:
sottoinsiemi di
oggetti

IDEA: Dato $\langle E, F \rangle$

- questa codifica delle proprietà degli insiemi in F

esempio (1)

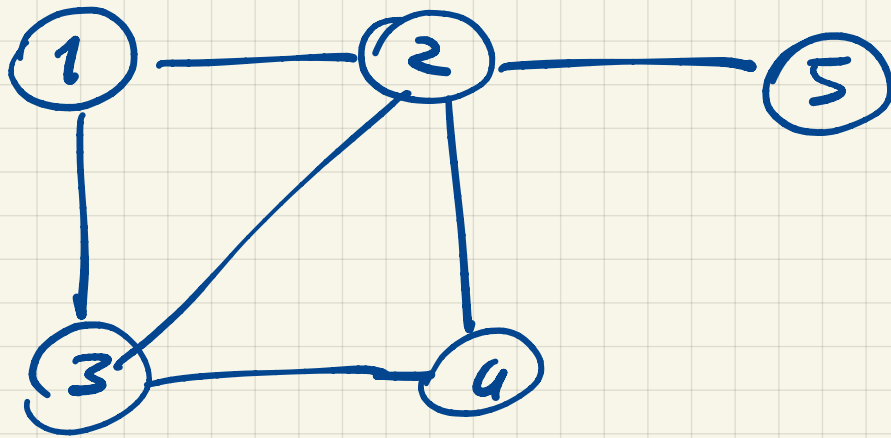
Dato $G = (V, E)$

$$E = \{e_1, e_2, \dots, e_m\}$$

costruiamo F come la famiglia dei sottoinsiemi di E che sono delle foreste $\Rightarrow A \in F,$

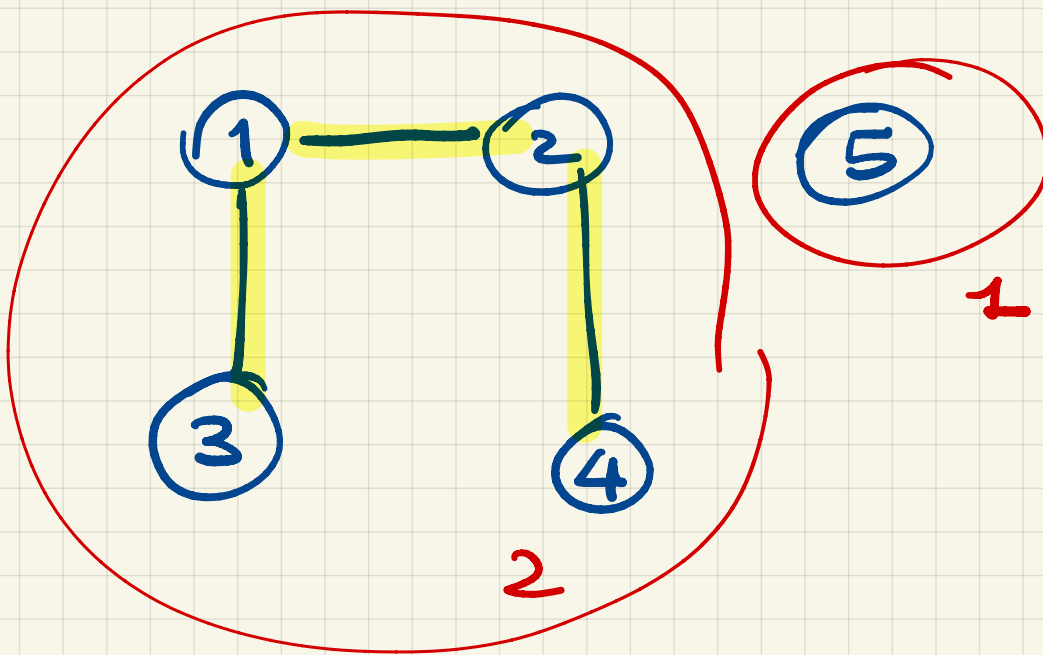
$$A \subseteq E,$$

A insieme di archi che
INDUCE su V una
foresta di alberi



$A_1 \Rightarrow$ foresta

$$A_1 = \{ (1, 2), (2, 4), (1, 3) \}$$



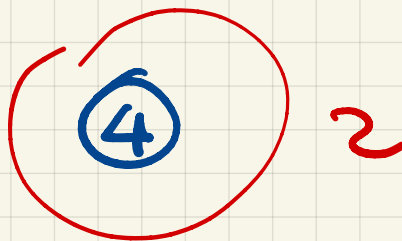
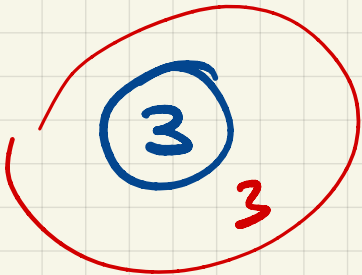
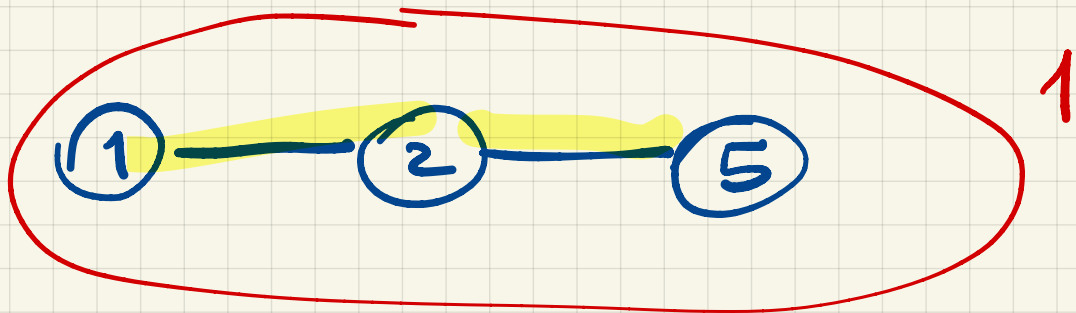
3 ardi

,

5 vertici

2 alberi

$A_2 \Rightarrow$ foreste

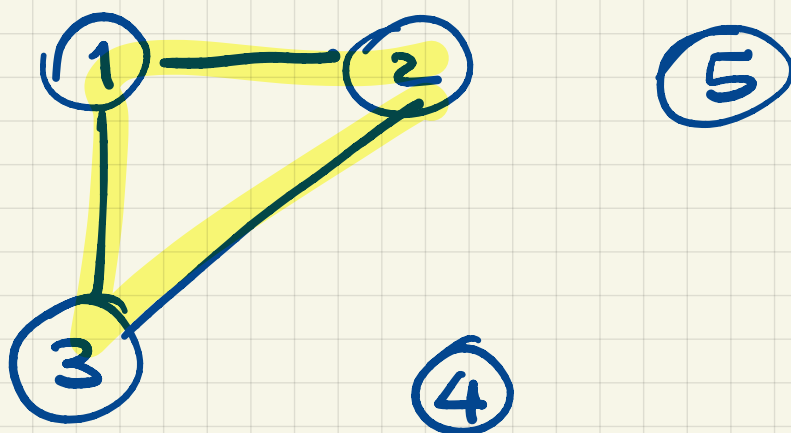


$$A_2 = \{ (1, 2), (2, 5) \}$$

2 ordi

5 vertici

3 alberi



$$A_3 = \{ (1,2), (2,3), (1,3) \}$$

non INDUCE

su V una
foresta

perché c'è il ciclo

UN ALBERO per definizione
è ACICLICO

"la FORESTA è un
insieme di alberi le cui
i vertici singoli del
grafo"

$\langle E, F \rangle$ codifica
della proprietà -

- E insieme di archi
di G
- F famiglia di sottinsiemi
di E che INDUCONO
foreste su G

$\Downarrow$ $\langle E, F \rangle$ fornisce una
CARATTERIZZAZIONE

delle SOLUZIONI:

PARZIALI (finali)
ad un problema
risolto da Greedy!

Domanda?

Dato una foresta

(A_i, V) quanti
alberi ha?

$$t = |V| - |A_i|$$

Lemme

Date una foresta indotta
da A_i sul grafo G ,
che indiciamo con le coppie

(A_i, V) , questa consiste
di t alberi che sono dati
come

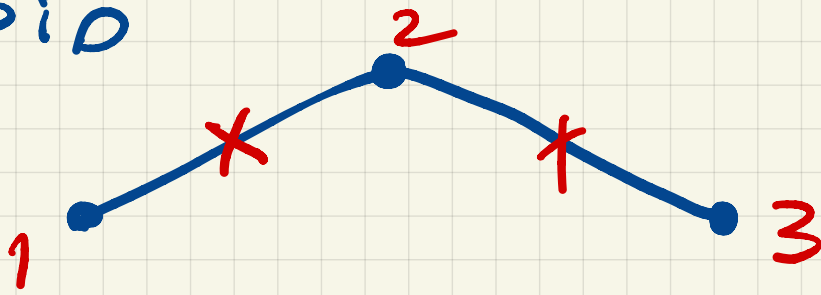
$$t = |V| - |A_i|$$

Dimostrazione

N.B. (PARENTESI) se ho
un albero con e_i archi
questo ha v_i vertici che
sono $e_i + 1$ cioè

$$e_i = v_i - 1$$

ESEMPIO



$$e_i = v_i - 1$$

l'albero
singolo

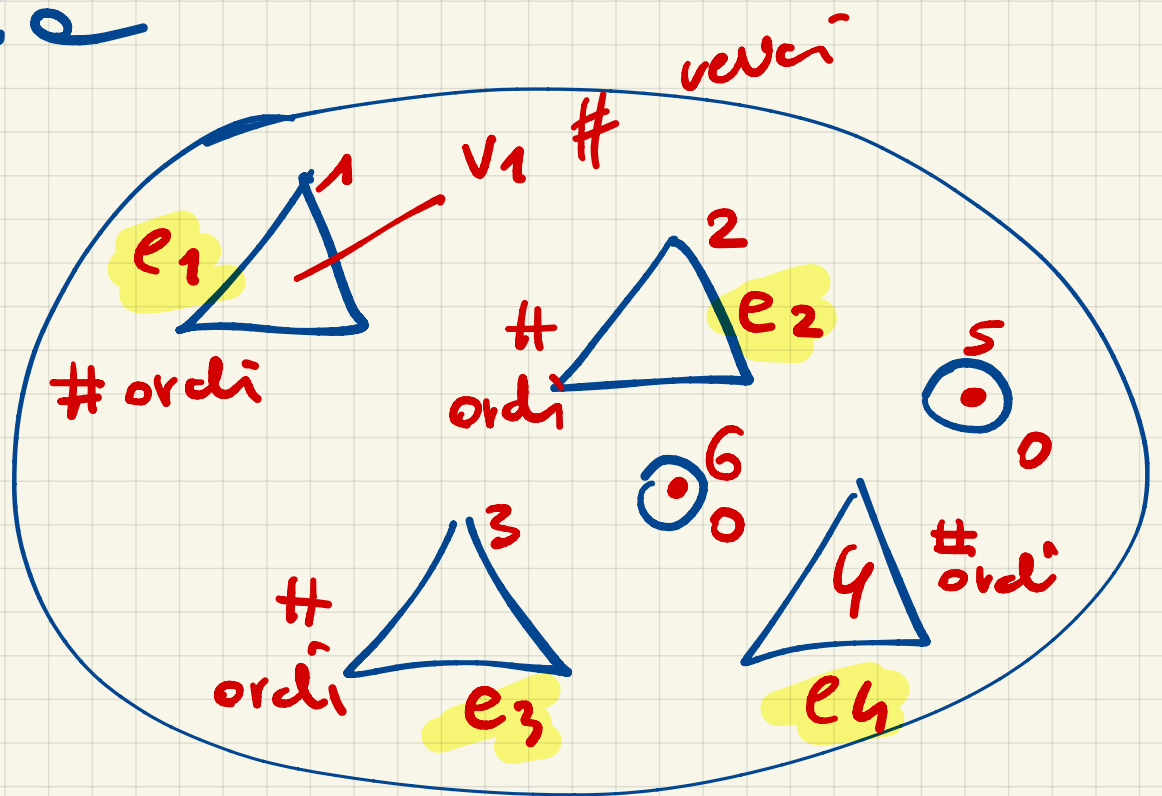
se ha e_i ardi deve
avere
cioè $e_i = v_i - 1$
 $e_i + 1 = v_i$

2 ardi $\rightarrow$ 3 vertici
 $e_i = 2$ $v_i = e_i + 1$

Siano e_i gli ardi
dell'albero i delle
foreste A_i ,
 v_i sono invece i vertici

dell'elberi i delle
foreste

A_i



$|A_i|$

$$\sum_{i=1}^t e_i$$

$$= \sum_{i=1}^t (v_i - 1) = \sum_{i=1}^t v_i - \sum_{i=1}^t 1 =$$

somma in A_i degli ordi
Total delle foreste (A_i, V)

$$= |V| - t$$

$$|A_i| = |V| - t$$

$$t = |V| - |A_i|$$

c.v.d

Ci serve poi per
 di mostrare che $\langle E, \bar{E} \rangle$
 definito sopra a parte
 del grafo $G = (V, E)$
 gode delle (1) proprietà

di ereditarietà (2) e scambio

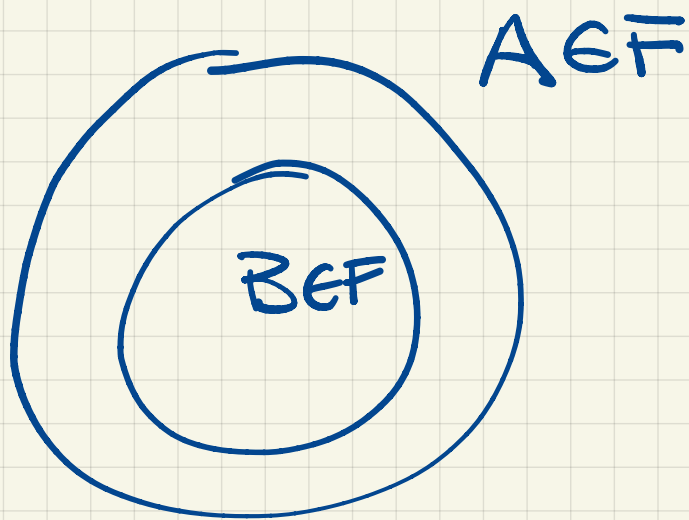
Def. UN SISTEMA di
indipendenza $\bar{e}$ è una
coppia $\langle E, F \rangle$ dove

- E $\bar{e}$ un insieme finito
- F $\bar{e}$ una famiglia
di sottoinsiemi di E ($F \subseteq 2^E$)

TALI CHE

$\forall A \in F, \forall B \subseteq A$
vale che $B \in F$

ereditarietà



AEF
(A sola
femmine)

A sottinsieme
della popolazione!

$B \subseteq A$ è di
sola femmine!

Esempio ①

Dato $G = (V, E)$

$\langle E, F \rangle$ dove E insieme
di archi di G

e dove F famiglia di
foreste di G allora

è un SISTEMA di

INDIPENDENZA dim.

data A generico $\in F$,

• A induce una foresta in G

$\forall B \subseteq A$, B ha meno
ordi di A per cui anche B

induce una foresta in G ,
perché se toglia ordi da
alberi ottengo ancora
alberi (NON CREO CICLI)

esercizio

Sia $\langle E, F \rangle$ tale che

$$F = \{ A \subseteq E : A \subseteq C \}$$

dove C è un insieme

fissato -

$\bar{E}$ un S.I.?

fisso

- $E =$ tutti gli uomini
- C (es. C è
l'universo dello
femmine
al mondo!)

$$F = \{ A \subseteq E : A \subseteq C \}$$

$\forall A$, se prendo B
qualsunque sottoinsieme di A
anche esso $\bar{E}$ sottoinsieme
di C per cui $B \in F$.

Def: INSIEME MASSIMALE
nella famiglia F
(non estendibile)

$A \in F$ è MASSIMALE
sse vale che

$$A \cup \{e\} \notin F,$$

$$\forall e \in E \setminus A -$$



soluzione
ottimale in
Greedy